💳 System Design: Online Payment System (Stripe-like)

🔷 *Step 1: Requirements, QPS Estimation, Design Goals*

⛳Functional Requirements:

- A buyer pays a seller for an item or service.
- Sellers receive a notification when a payment is completed
- Sellers can initiate refunds.
- Buyer and seller can view payment status and history.
- Card/bank information is tokenized and stored securely.
- Idempotent APIs for reliability.
- Webhooks are delivered reliably with retry mechanisms.

🎯 Non-functional Requirements

- High availability and fault tolerance
- PCI-compliant tokenization
- Reliable webhook delivery
- Immutable audit logs
- Scalability across regions
- Safe retries using idempotency keys

⚙️Estimated Peak QPS:

- 300K payments per day → ~3.5 QPS average, ~18 QPS peak
- 30K refunds per day → ~0.3 QPS average, ~1.5 QPS peak
- 5M read requests/day → ~60 QPS average, ~300 QPS peak
- 300K webhook deliveries/day → ~3.5 QPS average, ~18 QPS peak
- Total Peak QPS: ~350 QPS

🔷 *Step 2: High-Level Design*

✅Core Components:

- API Gateway: Authenticates and routes requests
- Payment Service: Handles buyer-to-seller payments and emits events
- Refund Service: Validates and processes refunds asynchronously
- Webhook Dispatcher: Delivers events to seller endpoints with retry/DLQ
- Tokenization Service: Secures sensitive data and returns tokens
- Ledger/Event Store: Stores immutable payment and refund lifecycle events
- Databases: PaymentDB, LedgerDB, BuyerDB, SellerDB

- External Payment Gateway: Interfaces with Visa/Mastercard, PayPal, etc.

🖊️Event Flow:

1. Buyer initiates payment → API Gateway → Payment Service
2. Payment Service validates, stores record, and contacts external gateway
3. On success:
    - Updates payment status
    - Publishes payment.success event
    - Webhook Dispatcher delivers event to seller
4. On failure:
    - Updates status
    - Publishes payment.failed event
    - No webhook sent

🔷 *Step 3: Core Components Design*

📑 API Design

API	Role
`POST /payments`	Buyer initiates a payment
`GET /payments/{id}`	Buyer/seller views payment status
`GET /buyers/{id}/payments`	Buyer views their transactions
`GET /sellers/{id}/payments`	Seller views income/payments
`POST /refunds`	Seller initiates refund
`GET /refunds/{id}`	Check refund status
`POST /tokenize`	Buyer tokenizes card/bank info
`POST /webhooks/register`	Seller registers webhook URL

≣Primary Data Models

1. Payment

Field Name	Type	Description
id	UUID	Unique payment ID
buyer_id	UUID	Reference to buyer
seller_id	UUID	Reference to seller
amount_cents	Integer	Amount in cents
currency	String	e.g. "USD", "EUR"
status	Enum	INIT, PENDING, PAID, FAILED, REFUNDED
payment_method_token	String	Tokenized card/bank ref
idempotency_key	String	For retry safety
created_at	Timestamp	Creation time
updated_at	Timestamp	Last status update

2. Refund

Field Name	Type	Description
id	UUID	Unique refund ID
payment_id	UUID	Refers to original payment
amount_cents	Integer	Refund amount
status	Enum	PENDING, SUCCESS, FAILED
created_at	Timestamp	Refund request time
updated_at	Timestamp	Refund result time

3.PaymentEvent (Ledger)

Field Name	Type	Description
id	UUID	Unique event ID

payment_id	UUID	Related payment
event_type	String	e.g. INITIATED, PAID, FAILED
payload	JSON	Metadata for the event
created_at	Timestamp	Event timestamp

🔷 *Step 4: Scaling Strategy and Bottleneck Resolution*

Scalability Tactics:

- Stateless services → horizontal scaling behind load balancers
- Shard PaymentDB and LedgerDB by seller ID or region
- Use Kafka for event stream processing
- Use Redis or DynamoDB for idempotency key caching
- Webhook Dispatcher supports retries and DLQs for resilience
- Token vault supports replicas and caching to reduce latency

Failure Recovery:

- Retry payment gateway failures with fallback strategies
- Circuit breakers for unstable external gateways
- Webhook retries with exponential backoff
- Eventual consistency for seller notifications
- Monitoring: log every webhook attempt and delivery status

Global Readiness:

- Route traffic regionally using geo-aware load balancers
- Store tokens in region-local vaults with master sync
- Use global ID generators (e.g. Snowflake) for distributed uniqueness